

Fashion Image Caption Generation

(with the use of CNN, Transformer, and Metadata)

Project Report

Presented by: Amrutha Jayachandradhara

Instructor: Prof. Amir Jafari, PhD

Institution: George Washington University

Abstract

E-commerce platforms maintain massive fashion catalogs where product descriptions are crucial for search relevance, user experience, and accessibility. Manual caption writing is expensive, slow, and often inconsistent, leading to generic and uninformative descriptions such as “Blue printed shirt for men” instead of “Men’s slim-fit blue printed cotton shirt, casual wear”. In this project, we build an automated fashion image captioning pipeline, beginning with a baseline encoder–decoder model using ResNet-50 and a Transformer decoder. We then extend this into a multimodal architecture (Model-1) that integrates image features with structured metadata via learnable embeddings and a fusion layer. We benchmark our models qualitatively against Qwen2-VL and implement an explainability module using gradient-based token importance. Our results show that metadata fusion significantly improves semantic accuracy, especially for garment type and gender, producing captions close to catalog-quality descriptions. A Streamlit interface demonstrates the models interactively.

1. Introduction

E-commerce platforms rely heavily on accurate and descriptive product captions for search optimization, user understanding, and accessibility. Manual caption writing is labor-intensive and inconsistent, especially at the scale of hundreds of thousands of products. This motivates the use of deep learning models to automate caption generation from product images and metadata.

Fashion products naturally contain multimodal information:

- Visual features: color, silhouette, texture, patterns;
- Structured metadata: gender, category, brand, season, article type.

Problem Statement: Given a product image and its metadata, generate an accurate, catalog-style caption and study how metadata fusion and large VLMs influence caption quality and interpretability.

Objectives:

1. Build an end-to-end data ingestion pipeline.
2. Train a baseline CNN + Transformer captioning model.
3. Extend to a metadata-fusion multimodal model (Model-1).
4. Benchmark against Qwen2-VL.
5. Add explainability using gradient-based token importance.
6. Deploy a Streamlit demo application.

2. Dataset and Preprocessing

2.1 Data Source

We use the publicly available Myntra Fashion Product Dataset from Kaggle, which consists of:

- Product images in JPEG format,
- Metadata table `styles.csv` with attributes: `id`, `gender`, `masterCategory`, `subCategory`, `articleType`, `baseColour`, `brand`, `season`, `year`, `usage`, and `productDisplay Name`.

2.2 Data Cleaning and Integration

Preprocessing includes:

1. Standardizing column names and dropping incomplete entries.
2. Linking each product ID to `images/<id>.jpg`.
3. Removing corrupted or unreadable images via PIL.
4. Creating a cleaned caption from `productDisplayName`, e.g.,
“Women Red Sleeveless Fit & Flare Dress” → “women red sleeveless dress”.

2.3 Train–Validation–Test Split

We split the dataset into:

- 70–80% training,
- 10–15% validation,
- 10–15% test.

Stratification ensures gender and categories are well balanced.

2.4 Vocabulary Construction

Captions are tokenized, rare words filtered, and a vocabulary is built. Special tokens: `<pad>`, `<bos>`, `<eos>`, `<unk>`.

3. Explainability

Understanding why a caption is generated is nearly as important as the caption itself, especially in applications where trust and transparency are required (e-commerce, catalog automation, or human-in-the-loop systems). To achieve this, we integrate an explainability module inside the Streamlit application. The module computes token-level importance for both of our trained models (Baseline and Model-1) and for Qwen2-VL. Importantly, the methods differ for the two model classes because we have access to gradients for our models, but not for Qwen.

This section describes in detail how these explainability scores are computed, how they are normalized, and how they appear in the application.

4.Qwen2-VL Model Implementation for Image Caption Generation

Technical Architecture

1. Environment Setup and Dependencies

- PyTorch: Deep learning framework for tensor operations and model execution
- PIL (Python Imaging Library): Image loading and preprocessing
- Transformers: Hugging Face library providing pre-trained model access

2. Device Configuration

- Automatically detects GPU availability for hardware acceleration
- Falls back to CPU if CUDA-enabled GPU is unavailable
- Critical for performance optimization (GPU is ~10-100x faster)

3. Model Initialization

```
MODEL_NAME = "Qwen/Qwen2-VL-2B-Instruct"
```

Model Selection:

- Qwen2-VL-2B-Instruct: 2 billion parameter vision-language model
- Instruction-tuned for following text prompts with visual inputs
- Optimized for multimodal understanding tasks

Processor Loading:

```
processor = AutoProcessor.from_pretrained(MODEL_NAME)
```

- Handles both text tokenization and image preprocessing
- Ensures inputs match the model's expected format
- Applies chat template formatting specific to Qwen2-VL

Model Loading:

```
model = AutoModelForVision2Seq.from_pretrained(  
    MODEL_NAME,  
    torch_dtype=torch.float32  
)  
.to(device)
```

- AutoModelForVision2Seq: Specialized architecture for vision-to-sequence tasks
- torch_dtype=torch.float32: Full precision for accuracy (32-bit floating point)
- .to(device): Transfers model to GPU/CPU

5. Caption Generation Pipeline

The generate_caption() function implements a 5-step inference pipeline:

Step 1: Image Loading

- Opens image file from disk
- Converts to RGB format (ensuring 3-channel consistency)
- Removes alpha channel if present

Step 2: Chat Message Construction

- Structured conversation format required by Qwen2-VL
- Multimodal content: combines image and text instruction
- Task-specific prompt: guides model to generate concise fashion descriptions
- Role-based architecture: mimics human-AI interaction

Step 3: Chat Template Application

- Converts structured messages into model-specific text format
- `add_generation_prompt=True`: Appends prompt tokens signaling the model should generate a response
- Creates properly formatted instruction string

Step 4: Input Processing

- Text tokenization: Converts text to token IDs
- Image processing: Resizes, normalizes, and converts image to tensors
- Tensor creation: `return_tensors="pt"` creates PyTorch tensors
- Device transfer: Moves all inputs to GPU/CPU

Step 5: Model Inference

- Autoregressive generation: Produces tokens sequentially
- `max_new_tokens=60`: Limits caption length (~12-15 words)
- Uses beam search or sampling for text generation
- Returns token IDs representing the caption

Step 6: Decoding

- Converts token IDs back to human-readable text
- `skip_special_tokens=True`: Removes `<pad>`, `</s>`, etc.
- `[0]`: Extracts first (and only) caption from batch
- `.strip()`: Removes leading/trailing whitespace

6. Execution Example

- Demonstrates usage with a sample fashion product image
- Outputs generated caption to console

Key Features and Advantages

1. Multimodal Understanding: Processes both visual and textual information simultaneously
2. Instruction Following: Responds to specific captioning instructions (concise, fashion-focused)
3. Pre-trained Knowledge: Leverages 2B parameters trained on diverse vision-language data
4. Flexible Architecture: Supports various vision-language tasks beyond captioning
5. Production-Ready: Uses stable Hugging Face APIs for deployment

Technical Specifications

- Model Size: 2 billion parameters
- Input Resolution: Automatically handled by processor (typically 448x448 or similar)
- Maximum Output Length: 60 tokens (~40-50 words)
- Precision: FP32 (float32)

- Memory Requirements: ~8-10GB GPU memory for inference
- Framework: PyTorch + Transformers

This implementation provides a modern, state-of-the-art approach to generating natural language descriptions of fashion products using vision-language models.



Unlike our PyTorch models, Qwen2-VL is used in *inference-only mode*. We cannot compute gradients through its layers inside Streamlit. Therefore, we use a hidden-state similarity explainability method.

Core Idea

Each generated token has a final-layer hidden representation:

$$h_i \in \mathbb{R}^d,$$

where d is the model's hidden dimension.

We also compute an aggregated caption representation (mean pooling):

$$h_{\text{agg}} = \frac{1}{L} \sum_{i=1}^L h_i.$$

A token is considered more important if its hidden representation aligns strongly with the “semantic center” of the caption:

$$\text{sim}(i) = \frac{h_i \cdot h_{\text{agg}}}{\|h_i\| \|h_{\text{agg}}\|}.$$

This cosine similarity yields values in $[-1, 1]$, which we map to the range $[0, 1]$ for visualization.

Interpretation

Tokens with high similarity values contribute more strongly to the caption's semantics. In practice:

Tokens like “Nike”, “splatter”, “logo”, “running” receive high scores. • Function words like “a”, “the”, “of” receive low scores.

This aligns with how LLMs encode semantics in high-dimensional embedding spaces.

Streamlit Display

- Color-coded caption with similarity heatmap,
- A comparison panel showing Model-1 importance vs. Qwen similarity.

This allows users to visually compare how a small multimodal model (Model-1) and a giant VLM (Qwen2-VL) prioritize different parts of the caption.

Why These Explainability Methods Are Meaningful For Baseline and Model-1

(Gradient Importance):

- Gradients quantify sensitivity of model output to token embeddings. • High gradient → small change in token → large change in logits. • This correlates with how much the decoder relies on that token.

This is similar to integrated gradients and saliency maps in NLP.

For Qwen2-VL (Token Similarity):

- Hidden-state similarity is the standard method for black-box LLM interpretation. • Cosine alignment indicates conceptual contribution to sentence semantics. • This matches attention patterns in transformer-based VLMs.

Summary

Our explainability module provides:

- Fine-grained token importance for Model-1 and Baseline using gradients. • Semantic token contribution for Qwen2-VL using cosine similarity. • A uniform visualization layer in Streamlit so users can compare models directly.
- These tools help users understand why the model selects certain descriptive words, how metadata influences Model-1, and how Qwen2-VL constructs semantically rich captions. Combined, they make the system transparent and interpretable.

Results and Discussion

Strengths:

- Robust data pipeline for multimodal fashion data.
- Clear improvement from Baseline to Model-1 due to metadata fusion. • Strong alignment between generated and ground-truth captions (BLEU/ROUGE). • Explainability makes model decisions more transparent.
- Streamlit app provides an interactive demo.

7. Limitations

- Vocabulary is limited; rare brands or styles may be mapped to <unk>.
- Visual explainability (e.g., Grad-CAM) is not integrated in the final app.
- Qwen2-VL comparison is primarily qualitative; a full quantitative benchmark is left for future work.

8. Future Work

- Use subword tokenization to better handle rare tokens and multilingual captions.
- Integrate Grad-CAM to visualize which image regions contribute most.
- Explore fine-tuning of VLMs such as Qwen2-VL on the fashion dataset.
- Add user feedback loops in the Streamlit app to iteratively refine captions.

9. Conclusion

We built an end-to-end multimodal image captioning system for fashion products, starting from a ResNet-50 + Transformer baseline and extending to a metadata-aware Model-1 architecture. Model-1 significantly improves BLEU and ROUGE scores and qualitatively produces more accurate, catalog-style captions. While large VLMs such as Qwen2-VL can be more confident and fluent, our smaller Model-1 effectively leverages structured metadata, making it highly suitable for real-world e-commerce deployments. The inclusion of explainability and an interactive Streamlit application further enhances the transparency and usability of the system.

10. References

- Myntra Fashion Product Dataset, Kaggle.
- He, K., Zhang, X., Ren, S., Sun, J. "Deep Residual Learning for Image Recognition." CVPR, 2016.
- Vaswani, A. et al. "Attention Is All You Need." NeurIPS, 2017.
- Qwen2-VL: Official model documentation and technical report.
- Selvaraju, R. R. et al. "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization." ICCV, 2017.